

Recruitment Management System

SYSTEM ARCHITECTURE & TECHNICAL DESIGN SPECIFICATION

Deliverable Documentation

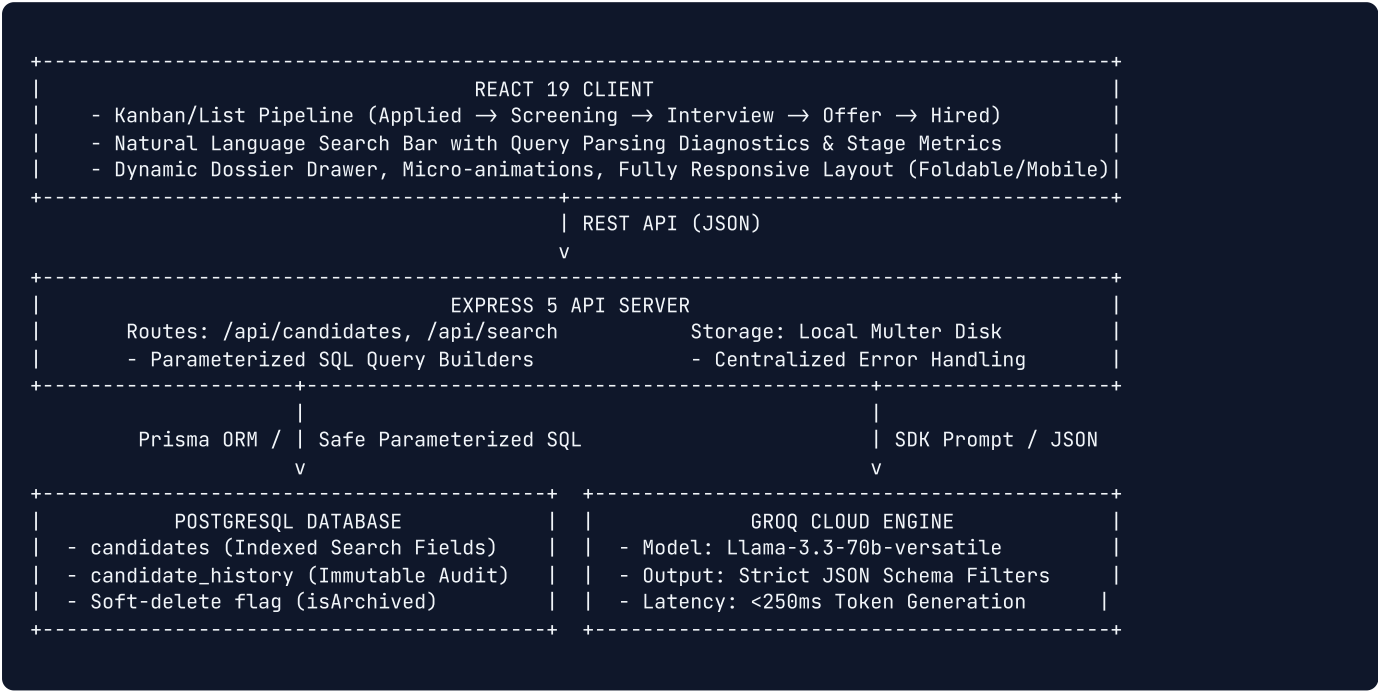
GitHub Repo:

github.com/Ayush/RecruitmentSoftware

Stack: React 19 • Express 5 • PostgreSQL • Groq Llama-3

1. ARCHITECTURE & COMPONENT INTERACTION

The Recruitment Management System follows a decoupled, resilient full-stack architecture built to manage applicant pipelines and execute natural language queries with sub-second response times.



Client Layer (React 19)

Built with Vite and TypeScript. Eliminates CSS frameworks in favor of handcrafted CSS tokens for fluid responsiveness across mobile devices, foldables, and desktop screens.

Server Layer (Node.js & Express 5)

Utilizes Controller-Service layering. Isolates route endpoints from database query generation and Groq LLM token streaming.

Persistence (PostgreSQL + Prisma)

Features an append-only candidate_history audit table with onDelete: NoAction to ensure legal audit trail retention across status changes.

AI Engine (Groq Llama-3.3-70B)

Fast LLM inference converting natural-language recruiter prompts into typed JSON filter objects consumed directly by database query builders.

2. ARCHITECTURAL DECISIONS & TECHNICAL TRADE-OFFS

Decision	Alternative Considered	Rationale
Groq Llama-3.3-70B Query Engine	Client/Server Regex heuristics, standard Gemini Flash, or local Python spaCy	Optimal Latency & Reasoning Sub-250ms response speed. Zero heavy local Python runtime dependencies. Discards brittle regex parsing while accurately interpreting colloquial recruiter terminology.
Immutable Audit History Table	Single updated_at timestamp or mutable status column	Compliance & Integrity Recruitment compliance requires non-repudiation. Every stage move (Applied → Screening → Interview → Offer → Hired) creates an immutable record.
Pure Vanilla CSS Design System	Tailwind CSS / Component Libraries (MUI/Chakra)	Precise Responsive Control Custom responsive grid and flex mechanics engineered for ultra-narrow mobile viewports and dual-screen foldables without CSS specificity bloat.
Disk-Based Resume Storage via Multer	PostgreSQL BYTEA / Base64 DB blobs	Storage Scalability Prevents database index bloat and high backup I/O overhead. Static asset serving via Express provides low latency.

3. AI COLLABORATION & KEY DISAGREEMENT CASE STUDY

Engineering Case Study: Rejecting Rule-Based Offline Regex Parsing

The AI Proposal: During the search engine implementation, the AI proposed a "Step 1: Offline Rule-Based NLP" layer that attempted to match keywords with regular expressions (e.g., `/(applied|screening|interview|offer|hired)/i`, `/(\d+)\+?\s*years?/i`) before delegating to the LLM.

"Don't want this Step 1: Offline Rule-Based NLP. Remove the changes of step 1 only." — Engineer Direction

Engineering Rationale for Disagreement:

- Semantic Ambiguity:** Conversational recruiter queries (e.g. *"candidates who did screening in frontend"*) trigger false positives with keyword matching (matching "screening" as candidate notes rather than stage status).
- Duplicate Maintenance Burden:** Maintaining a fragile regex parser alongside an LLM causes divergent parsing rules and edge-case fragility.
- Latency Reality:** With Groq's high-speed inference (<250ms latency), the LLM round-trip is fast enough for real-time querying, rendering rule-based pre-filtering obsolete.

4. FUTURE ROADMAP & WHAT WE'D BUILD WITH MORE TIME

1. Semantic Vector Search (pgvector)

Implement hybrid search combining structured SQL filters with dense embeddings over resumes and candidate interview notes.

2. Automated Resume OCR & Parsing

Deploy background workers using LLM vision/text extraction to automatically extract skills, contact details, and experience upon PDF upload.

3. Real-Time Pipeline WebSockets

Integrate Server-Sent Events (SSE) or Socket.io to synchronize candidate movements across collaborative hiring teams in real time.

4. Multi-Tenant Role-Based Access Control

Granular RBAC tiers (Admin, Hiring Manager, Recruiter, Reviewer) with organization-level data segregation.